TASK 1: BASIC NETWORK SNIFFER

Detailed Implementation, Architectural Overview, and Protocol Analysis Report

1. Executive Summary

This comprehensive technical document describes the engineering, structure, deployment, and practical testing of a **Basic Network Sniffer** engineered using Python and the Scapy library. Network sniffing constitutes an essential framework within cybersecurity and systems administration to intercept, parse, and review raw data moving across a network interface.

The primary operational objective of this project is to create an automated program capable of capturing packet sequences dynamically, exposing layered structural breakdowns, extracting transmission flows, and exposing packet metadata (including source/destination IPs, underlying communication protocols, and embedded application payloads). This analysis addresses network fundamentals, administrative privileges, defensive concerns, and programmatic decoding routines.

2. Architectural Foundations & Network Layer Theory

To grasp packet capturing mechanics completely, modern interactions must be related back to structural blueprints like the **OSI (Open Systems Interconnection)** model and the practical **TCP/IP stack**. When an application initiates data transmissions, data traverses downward through software layers, getting encased inside header bytes specific to each layer before hitting the physical network wire as a bitstream.

A packet sniffer hooks directly into the lower layers of this infrastructure. Typically, a standard Network Interface Card (NIC) discards any incoming frames whose destination hardware MAC address does not match its own. To capture broad network conversations, the sniffer must switch the interface into **Promiscuous Mode**. This tells the network adapter to forward every single received packet up to the Operating System's kernel registry, regardless of the destination endpoint specified in the layer-2 frame header.

> **The Encapsulation Mechanism:**
> [Application Data] → [TCP/UDP Header + Data] (Segment) → [IP Header + Segment] (Packet) → [Ethernet Header + Packet + Trailer] (Frame)

Our implementation intercepts these data segments at the Data Link/Network boundaries, parsing binary sequence fields into human-readable parameters using the architectural sequence below:

OSI Layer Name	Protocol Context	Captured Metrics & Parsed Structural Metadata
Layer 2: Data Link	Ethernet II, IEEE 802.3	Source and Destination Hardware MAC Addresses; Frame Type identifiers.
Layer 3: Network	IPv4, IPv6, ICMP, ARP	Source and Destination IP Addresses; TTL (Time to Live); Header Checksums.
Layer 4: Transport	TCP, UDP	Source and Destination Ports; Sequence Numbers, Acknowledgements, Flags (SYN, ACK, FIN).
Layer 5-7: Application	HTTP, DNS, FTP, SMTP	Raw application data payload text, query names, response status strings.

3. Comprehensive Python Source Code Implementation

The complete, standalone implementation of our packet analyzer is detailed below. The engine uses `scapy.all` to establish a native packet filtering socket hook, parses protocol layers sequentially, handles runtime errors, and flushes output buffers transparently.

```
#!/usr/bin/env python3
import os
import sys
from datetime import datetime
from scapy.all import sniff, IP, TCP, UDP, ICMP, Raw

def print_banner():
    """Displays initialization configurations to the shell terminal."""
    print("=" * 70)
    print("               PRODUCTION-GRADE PACKET CAPTURE ENGINE                ")
    print(f"  Initialized at : {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")
    print("  Target Scope   : Monitoring Inbound & Outbound Multi-Protocol Traffic")
    print("=" * 70)

def packet_callback(packet):
    """
    Invoked automatically for every intercepted packet matching filter criteria.
    Parses Layer 3 (IP) and Layer 4 (TCP/UDP/ICMP) structural components.
    """
    # Verify that the captured frame contains an IP layer component
    if packet.haslayer(IP):
        ip_layer = packet[IP]
        src_ip = ip_layer.src
        dst_ip = ip_layer.dst
        proto_num = ip_layer.proto

        # Map numeric protocol indicators to descriptive string identifiers
        proto_map = {1: "ICMP", 6: "TCP", 17: "UDP"}
        proto_name = proto_map.get(proto_num, f"UNKNOWN-{proto_num}")

        timestamp = datetime.now().strftime("%H:%M:%S.%f")[:-3]
        print(f"\n[{timestamp}] IP Packet: {src_ip} --> {dst_ip} | Layer 4 Protocol:
{proto_name}")

        # ---- SUB-ROUTINE: PARSE LAYER 4 TCP PROTOCOLS ----
        if packet.haslayer(TCP):
            tcp_layer = packet[TCP]
            print(f"  [TCP METADATA] Src Port: {tcp_layer.sport} | Dst Port: {tcp_layer.dport} |
Seq: {tcp_layer.seq} | Ack: {tcp_layer.ack}")
            print(f"  [TCP FLAGS]    Flags: {tcp_layer.underlayer.sprintf('%TCP.flags%')}")

        # ---- SUB-ROUTINE: PARSE LAYER 4 UDP PROTOCOLS ----
        elif packet.haslayer(UDP):
            udp_layer = packet[UDP]
            print(f"  [UDP METADATA] Src Port: {udp_layer.sport} | Dst Port: {udp_layer.dport} |
Length: {udp_layer.len}")

        # ---- SUB-ROUTINE: PARSE LAYER 4 ICMP PROTOCOLS ----
        elif packet.haslayer(ICMP):
            icmp_layer = packet[ICMP]
            print(f"  [ICMP METADATA] Type: {icmp_layer.type} | Code: {icmp_layer.code} |
Checksum: {icmp_layer.chksum}")

        # ---- SUB-ROUTINE: PAYLOAD DATA EXT_DECODING ----
        if packet.haslayer(Raw):
            raw_payload = packet[Raw].load
            print("  [PAYLOAD CONTEXT (Hexdump-Equivalent / Text)]")
            try:
                # Attempt decoding raw bytes into clean UTF-8 string format
                decoded_payload = raw_payload.decode('utf-8', errors='replace').strip()
```

```
                # Format large text lines elegantly into safe output block blocks
                formatted_lines = [decoded_payload[i:i+80] for i in range(0,
len(decoded_payload), 80)]
                for line in formatted_lines[:5]: # Limit to first 5 lines for clean logs
                    print(f"    | {line}")
                if len(formatted_lines) > 5:
                    print("    | ... [Truncated for visibility] ...")
            except Exception as ex:
                print(f"    | Raw Data Bytes (Hex Format): {raw_payload.hex()[:100]}...")

def main():
    """Main system orchestrator coordinating setup and execution controls."""
    # Security Check: Sniffing requires administrative kernel level system execution privileges
    if os.name != 'nt' and os.getuid() != 0:
        print("[!] CRITICAL ERROR: Superuser (root) privileges are required to run this
script.")
        print("    Please execute using: sudo python3 sniffer.py")
        sys.exit(1)

    print_banner()
    print("[*] Starting packet capture loop. Press Ctrl+C to terminate cleanly...")
    print("-" * 70)

    try:
        # Initiate Scapy sniffing module. store=0 ensures packets stream past memory cleanly.
        sniff(prn=packet_callback, store=0)
    except KeyboardInterrupt:
        print("\n[*] Termination command received. Flushing buffers and shutting down cleanly.")
        print("=" * 70)
    except Exception as e:
        print(f"[!] Runtime Exception Encountered: {e}")
        sys.exit(1)

if __name__ == "__main__":
    main()
```

4. Step-by-Step Structural Logic Breakdown

The code is organized into modular segments to isolate core sniffing engines from parsing and presentation mechanics:

- **Administrative Validation Check:** Network sniffers interact directly with the physical network adapter via kernel-level system sockets. On Linux distributions, this requires root privileges. The script checks `os.getuid() != 0` to prevent unprivileged execution errors.
- **The Scapy Sniffing Loop Core:** The `sniff()` operation drives the engine. By defining `store=0`, captured packets are immediately released from system memory after handling. This prevents memory leaks and script crashes during long capture sessions on busy networks.
- **Layer Verification & Extraction:** Packets are filtered via conditional checks like `packet.haslayer(IP)`. This shields the parser from malformed packets, loopbacks, or non-IP frames that would otherwise trigger runtime exceptions.
- **Application Payload Recovery:** The `Raw` layer data represents the raw text or binary payloads sent across the wire. The script safely decodes these bytes using `errors='replace'` to prevent broken characters from interrupting execution.

5. Simulated Session Logs & Comprehensive Protocol Analysis

The following simulated transaction log illustrates how the sniffer processes and displays common internet traffic, including an HTTP request, a DNS lookup, and an ICMP query.

```
======================================================================
               PRODUCTION-GRADE PACKET CAPTURE ENGINE
  Initialized at : 2026-06-12 14:30:15
  Target Scope   : Monitoring Inbound & Outbound Multi-Protocol Traffic
======================================================================
[*] Starting packet capture loop. Press Ctrl+C to terminate cleanly...
----------------------------------------------------------------------

[14:30:22.104] IP Packet: 192.168.1.45 --> 93.184.216.34 | Layer 4 Protocol: TCP
  [TCP METADATA] Src Port: 51234 | Dst Port: 80 | Seq: 310495831 | Ack: 0
  [TCP FLAGS]    Flags: S

[14:30:22.142] IP Packet: 93.184.216.34 --> 192.168.1.45 | Layer 4 Protocol: TCP
  [TCP METADATA] Src Port: 80 | Dst Port: 51234 | Seq: 984512314 | Ack: 310495832
  [TCP FLAGS]    Flags: SA

[14:30:22.145] IP Packet: 192.168.1.45 --> 93.184.216.34 | Layer 4 Protocol: TCP
  [TCP METADATA] Src Port: 51234 | Dst Port: 80 | Seq: 310495832 | Ack: 984512315
  [TCP FLAGS]    Flags: A
  [PAYLOAD CONTEXT (Hexdump-Equivalent / Text)]
    | GET /index.html HTTP/1.1
    | Host: www.example.com
    | User-Agent: Mozilla/5.0 (X11; Linux x86_64)
    | Accept: text/html
    |

[14:30:25.412] IP Packet: 192.168.1.45 --> 8.8.8.8 | Layer 4 Protocol: UDP
  [UDP METADATA] Src Port: 42119 | Dst Port: 53 | Length: 37
  [PAYLOAD CONTEXT (Hexdump-Equivalent / Text)]
    |
\x12\x34\x01\x00\x00\x01\x00\x00\x00\x00\x00\x00\x03api\x06github\x03com\x00\x00\x01\x00\x01

[14:30:28.910] IP Packet: 192.168.1.45 --> 192.168.1.1 | Layer 4 Protocol: ICMP
  [ICMP METADATA] Type: 8 | Code: 0 | Checksum: 24321
```

Detailed Traffic Assessment

1. **TCP Handshake Mechanics (HTTP Traffic):** The first three log entries capture a classic 3-way TCP handshake (SYN → SYN-ACK → ACK). The flags correspond directly to connection states (`S` = Synchronize, `SA` = Syn-Ack, `A` = Acknowledge). The third packet reveals a cleartext HTTP GET request payload, highlighting how easily unencrypted web traffic can be monitored.
2. **UDP Stateless Operations (DNS Request):** The fourth entry displays a standard UDP interaction targetting DNS port 53. Because UDP is connectionless, there are no structural flags or handshakes. The payload contains raw DNS query strings seeking IP resolution for `api.github.com`.
3. **ICMP Diagnostics:** The final log captures an ICMP Echo Request (Type 8, Code 0), typically generated by the `ping` utility to verify network layer availability between endpoints.

> **Security & Data Privacy Implications:**
> The captured HTTP and DNS logs illustrate how cleartext traffic leaks sensitive internal states to onlookers. This demonstrates why production services mandate switching from legacy cleartext protocols to modern encrypted variants like HTTPS (TLS) and DNS-over-HTTPS (DoH). Encrypted payloads mask underlying session data, rendering intercepted bits unreadable to casual sniffers.

6. Deployment, Verification & Operational Guide

To deploy and run this script within an administrative sandbox or network engineering lab, follow the structured setup instructions outlined below.

Step 1: Environmental Setup & Dependency Provisioning

Ensure Python 3 and the system package manager `pip` are available on your platform. Install Scapy using the terminal:

```
$ pip install scapy
```

Note: On Linux platforms, you may need to install libpcap dependencies explicitly using your native package manager (e.g., `sudo apt install libpcap-dev`) to facilitate reliable low-level raw socket operations.

Step 2: Execution and System Execution Controls

Save the provided script file as `sniffer.py`. Grant execution permissions and run the script with elevated root privileges:

```
$ chmod +x sniffer.py
$ sudo ./sniffer.py
```

Step 3: Generating Test Traffic

To verify your sniffer is working properly, open a separate terminal window and generate sample network traffic using standard tools like `curl` or `ping`:

```
$ ping -c 2 192.168.1.1
$ curl http://www.example.com
```

The sniffer window will update in real time as these operations trigger packet exchanges, confirming successful end-to-end packet processing.

7. Conclusion and Security Policy

This network sniffer project demonstrates the foundational principles of network traffic observation and inspection. By intercepting packets at the data link boundary, parsing their layered structures, and decoding underlying payload fields, we gain clear insight into how data traverses enterprise systems.

While sniffing is indispensable for network diagnostic auditing and performance troubleshooting, it also highlights the security risks of unencrypted protocols. Deploying encryption across all application layers remains the absolute defensive baseline for safeguarding modern network systems against unauthorized interception.